

Bash Class: Loops & Conditionals with Real Genomic Data (Human Chromosome 21)

Today we're going to practice more basic shell commands. To do this, we'll start working with real biological data, so you can begin getting familiar with manipulating and understanding biological data.



Remember to document each step in a separate script document, following the best practices we covered last class.

Chromosome 21

We're going to work with human Chromosome 21. It's the smallest human autosome, and it was the second chromosome to be fully sequenced in the Human Genome Project. It has approximately 215–234 protein-coding genes.

It's of significant medical importance due to its link to Down syndrome (trisomy 21). It houses critical loci like the *APP* gene, whose extra copy directly connects trisomy 21 to early-onset Alzheimer's disease, alongside genes like *SOD1*, which regulates oxidative stress and cellular aging.

Activity goals:

- Practice creating and moving around directories.
- Download data from NCBI
- Practice file manipulation
- Learn basic shell commands for data manipulation.
- Understand loops and conditionals.
- Understand the structure of a GTF file.
- Get familiar with fasta files.
- Practice script documentaton. Last class we discussed that a proper managment of biological databases should follow the FAIR principles, and it needs to be **F**indable and **A**ccesible. Repositories like Ensembl, the UCSC database, and NCBI are one of the major public repositories commonly used.

One of the most common files that you will be using when working with biological data, is the **annotation** file. It is usually represented in a 9-column format file [GFF or GTF](#) containing information about every gene, transcript, isoforms, psedugenes, variants, non-coding regions, etc. together with metadata such as their accession number.

This is from [ensembl description](#):

"Fields must be tab-separated. Also, all but the final field in each feature line must contain a value; "empty" columns should be denoted with a '.' *

1. **seqname** - name of the chromosome or scaffold; chromosome names can be given with or without the 'chr' prefix. Important note: the seqname must be one used within Ensembl, i.e. a standard chromosome name or an Ensembl identifier such as a scaffold ID, without any additional content such as species or assembly. See the example GFF output below.

2. **source** - name of the program that generated this feature, or the data source (database or project name)
3. **feature** - feature type name, e.g. Gene, Variation, Similarity
4. **start** - Start position of the feature, with sequence numbering starting at 1.
5. **end** - End position of the feature, with sequence numbering starting at 1.
6. **score** - A floating point value.
7. **strand** - defined as + (forward) or - (reverse).
8. **frame** - One of '0', '1' or '2'. '0' indicates that the first base of the feature is the first base of a codon, '1' that the second base is the first base of a codon, and so on..
9. **attribute** - A semicolon-separated list of tag-value pairs, providing additional information about each feature.

The annotation file usually contains only information about specific features of gene, not the fasta sequence.

For todays class exercise, we will:

- Download the human genome annotation (gtf file, the whole genome, all chromosomes).
- Extract only the information for chromosome 21.
- Extract the gene names and accession numbers on chromosome 21.
- Randomly select 20 genes and download their actual sequences from NCBI, in a single loop.
- Create and document your script.

[!TIP] I added the basic syntax for the commands that we are using today so it is easier for you to revisit and to create your own dictionary. Even if the **dictionary** does not count for your final grade, I will considered it for an extra point at the end of the semester.

1. Create your working directory

Last class we created our main `btec_640` working directory.

👉 Open your terminal, make sure is in `bash`

👉 Go to your `/btec_640/class_excercises` directory that you created last class.

👉 Create a new directory for todays class (named it following best practices)

Paste your code below:

```
#Paste here:nomth@Nomthandazo MINGW64 ~/btec_640/btec-640/Class-exercise/Antigen
nomth@Nomthandazo MINGW64 ~/btec_640/btec-640/Class-exercise/Antigen
```

2. Download the chr21 annotation file

To download a file from the web we can use `wget` or `curl`. Today we will be using `curl`.

[!TIP] **Syntax:** `curl`

```
curl -o OUTPUT_FILENAME "URL"
```

- `curl` fetches whatever is at a URL.
- `-o` tells it to *save* what it fetches into a file, instead of just printing it to your screen. Always follow `-o` with the filename you want.
- The URL goes in quotes so the shell doesn't get confused by special characters inside it.

1. Check that you are inside the working directory for today's activity `pwd`
2. We are going to download the human genome annotation. This will be your input data. Create the `input_data` directory and **move** to that directory.
3. Once inside your "input_data" directory copy and paste this command:

```
curl -o hg38.ncbiRefSeq.gtf.gz  
"https://hgdownload.soe.ucsc.edu/goldenPath/hg38/bigZips/genes/hg38.ncbiRefSeq.gtf.gz"
```

This is the **whole human genome's** annotation but we just need chromosome 21. Try to open the file with `less`

[!TIP] **Syntax:** `less`

```
less FILENAME
```

Code:

```
less hg38.ncbiRefSeq.gtf.gz
```

Well, it seems that we need to unzip it, for this we will use `gunzip`

[!TIP] **Syntax:** `gunzip`

```
gunzip FILENAME.gz
```

- `.gz` is the gzip compression format. `.zip` is a *different* format — `unzip` will not work on a `.gz` file, and will error out. The file extension tells you which decompression tool to use.
- `gunzip` replaces the `.gz` file with the uncompressed version, same name minus `.gz`.

Code:

```
gunzip hg38.ncbiRefSeq.gtf.gz
```

let's open it again (**NOTICE THAT THE FILENAME CHANGED**):

- `space` = page down, `q` = quit.

While scrolling, identify each of the 9 GTF columns.

Now try `head` and `tail` commands on the same file, see what happens?

Lets create our `analysis` directory. Go back to your workdir and create the `analysis` directory.

Do a soft link of the unzipped file:

```
ln -s ../input_data/hg38.ncbiRefSeq.gtf
```

3. Extract the names and accession numbers from all the protein-coding genes of chromosome 21 in humans

? We just need the protein-coding genes from chromosome 21. Forget about the command line, what is the logical workflow or syntax that you would use to extract only the information for chromosome 21?

Type your answer:

Step 1:

Start with the GTF file containing all genes

Filter by chromosome 21 , keep only records where the chromosome column is chr21.

Filter by gene type , from those chromosome 21 records, keep only entries where the gene type is protein_coding

Extract the desired information , such as gene name, gene ID, chromosome, start/end position, etc.

Output the filtered records into a new file

There are many ways to do this, this example is just one of them. I encourage you to explore your own solution if your reasoning was different from the one we are going to work today:

The workflow would be:

3a. Extract all lines that contains chr21

But how do we know this information? Is it actually there? For this we use one of the most common commands in shell: **grep**. The grep (Global Regular Expression Print) command is use to search for specific words, phrases.

[!TIP]Syntax: grep

```
grep -FLAG "PATTERN" FILENAME
```

- Flags are optional, you will get familiar with them the more you practice.

Let's see if **chr21** is present in our file, and how many lines are in from chr21 only in the GTF file?

Code

```
grep "chr21" hg38.ncbiRefSeq.gtf
```

It is impossible to count right? Let's try with the "count" flag.

```
grep -c "chr21" hg38.ncbiRefSeq.gtf
```

? Question: What is the total line count for a GTF file filtered for chr21?

Type your answer:
52142

Ok, chr21 is there, let's print again all the chr21 but now we are going to save it in a new file named: **chr21.gtf**. After doing this explore your file.

```
grep "chr21" hg38.ncbiRefSeq.gtf > chr21.gtf
```

? What would happen if you keep the `grep -c` flag and save it as chr21.gtf?

Type your answer:

```
grep "chr21" ...      → gives you the actual matching lines
grep -c "chr21" ...   → gives you the number of matching lines
```

3b. Extract only the gene name and accession numbers for protein-coding genes.

? Let's get familiar with our GTF file, in which column is this information?

Type your answer:

9

We want to keep only **protein-coding genes**. This is why is so important the NCBI accession number.



From [NCBI website](#):

NCBI Reference Sequence accession numbers (or RefSeq accessions) uniquely identify sequence records that NCBI derives from selected GenBank records. GenBank is a highly redundant database. Hence, NCBI creates RefSeqs to provide a less redundant representation of the naturally occurring nucleic acid and protein molecules.

The generic format of a RefSeq accession is as follows:

[two-letter alphabetical prefix][_][series of digits or alphanumeric characters][.][version number]

And this is the description of each two-letter prefix:

Prefix	Molecule type	Description
AC_	Genomic	Complete genomic molecule, usually alternate assembly
NC_	Genomic	Complete genomic molecule, usually reference assembly
NG_	Genomic	Incomplete genomic region
NT_	Genomic	Contig or scaffold, clone-based or WGSa

Prefix	Molecule type	Description
NW_	Genomic	Contig or scaffold, primarily WGSa
NZ_b	Genomic	Complete genomes and unfinished WGS data
NM_	mRNA	Protein-coding transcripts (usually curated)
NR_	RNA	Non-protein-coding transcripts
XM_c	mRNA	Predicted model protein-coding transcript
XR_c	RNA	Predicted model non-protein-coding transcript
AP_	Protein	Annotated on AC_ alternate assembly
NP_	Protein	Associated with an NM_ or NC_ accession
YP_c	Protein	Annotated on genomic molecules without an instantiated transcript record
XP_c	Protein	Predicted model, associated with an XM_ accession
WP_	Protein	Non-redundant across multiple strains and species

So we actually don't need all the columns, and we just need the **NM_ prefix**

Our final table should look like this:

Gene	Accession number
gene1 name	accession number
gene2 name	accession number
gene3 name	accession number

Problem: predicted vs. curated accessions. NM= curated by RefSeq staff. XM_/XR_ = computationally predicted, not experimentally confirmed. For this class we only want curated accessions.

We can easily do this with **grep** again:

```
grep "NM_" chr21.gtf > refseq_chr21.gtf
```

Count again how many genes you have now, and compare with the non-filtered file.

Now let's remove all the information that we don't need. Your file should look something like this:

```
chr21_KI270872v1_alt    ncbiRefSeq.2022-10-28    transcript    49816    80944    .    -  
.    gene_id "LSS"; transcript_id "NM_001145436.2"; gene_name "LSS";  
chr21_KI270872v1_alt    ncbiRefSeq.2022-10-28    exon    49816    52605    .    -    .  
gene_id "LSS"; transcript_id "NM_001145436.2"; exon_number "17"; exon_id  
"NM_001145436.2.17"; gene_name "LSS";
```

This is a lot, we just need the information in column 9, for this, lets use **awk**:

[!TIP] **Syntax: awk (basic field filtering)

```
awk -F 'DELIMITER' 'CONDITION{FIELDS}' FILENAME
```

awk is for pattern scanning and text processing. It processes text line-by-line, breaks each line into columns. It reads a file **one line at a time** and applies your condition to every line.

- **-F** is a flag that sets the delimiter used to split each line into fields. Default is whitespace; here our file is tab-separated, so **-F '\t'**.
- **CONDITION** is checked on every line. If it's true, the line prints.
- **FIELDS** are referred to as **\$1, \$2, \$3...** (**\$1** = first column, etc.)

Let's see how the column where the gene name and accession number looks like:

Code:

```
awk -F '\t' '{print $9}' refseq_chr21.gtf | head
```

But we still have a lot of information that is not useful right now. In the example below the gene name is **LSS** and the accession number is **NM_001145436.2**

```
gene_id "LSS"; transcript_id "NM_001145436.2"; exon_number "17"; exon_id  
"NM_001145436.2.17"; gene_name "LSS";
```

? Question: Using **awk**, what would you do to split column 9, don't type the code, just your logic.

Type your answer:

Look at column 9 of the GTF file.

Column 9 contains several pieces of information together, such as the gene name, accession number, gene ID, etc.

Use **awk** to split column 9 using its delimiter (the character that separates those pieces of information).

This creates separate fields that you can then select individually.
From those separated fields, you can extract the gene name and accession number

Column 9 looks like: `gene_id "APP"; transcript_id "NM_000484.4"; ...`

If we tell awk to split on the quote character instead of tabs, the gene name and accession fall neatly into their own numbered fields:

```
gene_id "  APP  "; transcript_id "  NM_000484.4  "; ...
$1      $2      $3      $4
```

So `$2` = gene name, `$4` = transcript accession.

Code

```
awk -F '\t' '{print $9}' refseq_chr21.gtf | awk -F '"' '{print $2, $4}' | head
```

Problem: one gene, many transcript rows. A single gene has one row per transcript/exon, so a naive extraction repeats the gene name many times (and if a gene has multiple splice variants, multiple *different* accessions).

Syntax: the `!seen[]++` pattern (keep only the first occurrence)

```
awk -F '\t' '{print $9}' refseq_chr21.gtf | awk -F '"' '!seen[$2]++ {print $2, $4}' refseq_chr21.gtf > gene_accession.txt
```

Check it:

```
wc -l gene_accession.txt
head gene_accession.txt
```

4. Download the sequence of 10 genes.

Let's pick the first 10 genes to get their corresponding fasta sequences. For this we will use `head`

Code:

```
head -n 10 gene_accession.txt > 10_genes.txt
cat 10_genes.txt
```

Now let's loop over the gene list and download each one

[!TIP] **Syntax: `while read` (loop over a file, line by line)**

```
while read -r var1 var2
do
    COMMANDS_USING_${var1}_AND_${var2}
done < INPUT_FILE
```

- `while read -r var1 var2` reads one line at a time from whatever is fed into it. If the line has two space-separated values, they're automatically split into `var1` and `var2`.
- Everything between `do` and `done` is the loop body — it runs once for every line in the input file.
- `< INPUT_FILE` at the very end feeds the file into the loop, one line per iteration.

Syntax: `if / else` (conditional)

```
if [ CONDITION ]
then
    COMMANDS_IF_TRUE
else
    COMMANDS_IF_FALSE
fi
```

- `if [ CONDITION ]` checks something. Note the required spaces inside the brackets: `[ CONDITION ]`, not `[CONDITION]`.
- `then` starts what happens if the condition is true.
- `else` (optional) is what happens if it's false.
- `fi` closes the `if` block (it's "if" spelled backwards — bash's convention for closing block keywords).
- `[ -s "$file" ]` is a specific, common condition: "does this file exist **and** is it non-empty" — exactly what you want to check right after a download.

Full loop:

```
while read -r gene accession
do
    curl -o "${gene}.fasta"
    "https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?
db=nucleotide&id=${accession}&rettype=fasta&retmode=text"
```

```
done < 10_genes.txt
```

Check the results:

```
ls -l *.fasta
```

To get credit:

Create a document with all the code that you use today. The description of each step and save it as `download_chr21.sh`.

Upload this document to GitHub in a new directory named `class_exercises` within your `btec_640` repository, along with the document containing your answers.